# Spatial Distance Histogram Parallel Optimizations

Matthew West

## 1 Introduction

Spatial Distance Histograms (SDH) are a graphical representation of the distribution of distances among a set of points. They can be used to characterize the clustering of data points in many applications. Because the distances between every point must be calculated, creating an SDH is a $\Theta(n^2)$ operation, where n is the total number of points.

This calculation can be expedited by running it in parallel on a GPU. The naive implementation, while vastly quicker than running on the CPU, is far from optimal, as seen in the many optimization methods in Kumar, et al.[1]. The rest of this report is dedicated to exploring the optimizations used in the improved kernel and the results they achieved.

## 2 Optimizations

### 2.1 Output Privitization

Output privatization is a parallel computing optimization technique where multiple sub-copies of the output are computed to reduce the number of memory access conflicts. When multiple threads attempt to access the same memory address, they must wait to sequentially perform their operation.

This technique led to the greatest single speedup. With a single private copy made for each block of threads, there was about a 2x speedup.

#### 2.1.1 Multiple Private Copies

The next logical step from one private copy is to create multiple private copies. However, creating the maximum number of copies is unfeasible. The more copies that are created, the fewer blocks that can be run on a multiprocessor at the same time.

The final kernel uses a model to determine the best number of copies to create. It estimates memory access speed and the number of rounds required for each number of shared histograms. The model is based on the model described in a paper on 2-body statistics by N. Pitaksirianan et al. However, during testing, the model did not weigh the cost of a conflict enough. This led to it creating fewer copies than was optimal in certain scenarios. To remedy this, the conflict cost was weighted more.

To further increase the number of possible copies, the size of each private copy was reduced. The maximum size of each bucket was reduced since each private copy counter will not need to hold the final value of the histogram.

#### 2.1.2 Parallel Reduction

Since multiple sub-copies of the output were created, they must be combined into the final output. The optimized kernel first combines the local copies of each block into a single local copy. It does this through a parallel reduction. Each block then adds its local copy into the final output copy.

This would likely be improved by copying all of the reduced block copies from shared into global, then reducing in parallel there. In a partial implementation, it increased performance by about 4%. However, this was not made fully working in time.

### 2.2 Tiling

Tiling is an optimization technique where values that will be repeatedly used will be stored in faster memory to reduce the cost

of accessing global memory. The input data can be divided into sections. Every element of this section must compute its distance from each anchor value in the block. This brought a modest speed increase to the kernel.

#### 2.2.1 Intra-Tile Distance Balancing

On the step for computing the distances of points within the tile, there is control divergence. The result of this is that some resources will be reserved but not used. A partially working version was implemented and saw a marginal speed increase. This was also not able to be made fully working in time.

### 2.3 Memory Coalescing

When something in memory is accessed, it becomes quicker to access the adjacent memory. Making all the threads access sequential elements in the input data can improve the access times. This also led to a marginal speed increase.

### 2.4 Other Optimizations

A tweak that saw a noticeable performance increase is running with a block size that is close to the number of output buckets. So for running with a bucket width of 500 (80 bins), 64 threads will work the best. Another optimization that proved to be worthwhile was to change the square root function from the standard double precision to a double precision reciprocal square root and then multiply that by the starting value.

Table 1: Time (in seconds) of each implementation and speedup from Project 1 to 2

| Number of Particles | CPU | Project 1 | Project 2 | Speedup |
|---|---|---|---|---|
| 10k | 1.355137 | 0.044340 | 0.015016 | 2.95x |
| 100k | 135.4629 | 3.235683 | 0.977567 | 3.31x |
| 500k | 3386.080 | 83.29670 | 25.58086 | 3.26x |

## 3 Results

The CPU, naive kernel (Project 1), and the optimized kernel (Project 2) were all run with 64 threads per block and a bucket size of 500. As shown in Table 1, there was around a 3x speedup from the optimizations.

## 4 Conclusion

The next steps to further optimize the kernel would be to finish the improved parallel reduction and the intra-block distance balancing. However, they will likely only improve the speed by a small amount.

**Note: The recomended block size for 500 bucket width is 64 threads**
See the full source code at github.com/mwest17/spatial-distance-histogram

## References

[1] Anand Kumar, Vladimir Grupcev, Yongke Yuan, Jin Huang, Yi-Cheng Tu, and Gang Shen. Computing spatial distance histograms for large scientific data sets on-the-fly. *IEEE Transactions on Knowledge and Data Engineering*, 26(10):2410–2424, 2014.